

Mixture of Experts Models

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

Mixture of Experts Models

Scaling capacity without scaling compute

1. Motivation
2. Learning Outcomes
3. What is a Mixture of Experts (MoE)?
4. Sparse vs. Dense Models: Trade-offs & Motivations
5. Gating Mechanisms – How Experts Are Selected
6. Routing in MoE
7. MoE at Scale
8. Benefits of MoE
9. Limitations of MoE Models
10. References & Further Reading

► Why study Mixture of Experts?

- Scaling dense LLMs is costly: every parameter participates in every forward pass, so capacity and compute grow together.
- **MoE models** break that link: massive total capacity with far lower computation per token, because only a few experts activate per input.
- Many frontier systems are MoE models (e.g. DeepSeek-V3/R1 with 671B total but 37B active parameters; Mixtral; Switch Transformer).

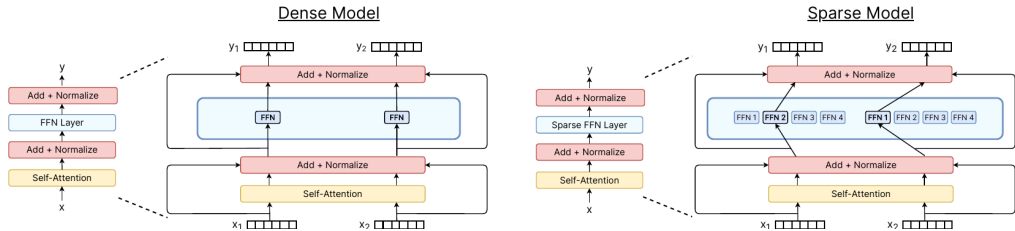
► Example:

- *A dense 671B model would be prohibitively expensive to serve; DeepSeek-R1 routes each token through a small subset of experts, giving 671B-scale capacity at roughly the inference cost of a 37B model.*
- Large Reasoning Models build directly on these architectures: several leading reasoning systems are trained and served as MoE models.

By the end of this session, you should be able to:

- ▶ Explain the core idea of Mixture-of-Experts (MoE) architectures and why they decouple model capacity from per-token compute.
- ▶ Contrast sparse and dense models and reason about their trade-offs.
- ▶ Describe gating mechanisms, top- k routing, and noisy gating.
- ▶ Explain load balancing, the auxiliary loss, and expert-collapse failure modes.
- ▶ Identify how MoE layers integrate into Transformers (Switch Transformer, GShard, Mixtral, DeepSeek) and how expert parallelism distributes them.
- ▶ Assess the benefits and limitations of MoE models in practice.

What is a Mixture of Experts (MoE)?



A dense model sends every token through the same FFN; a sparse expert model routes each token to a subset of the expert FFNs. Source: Fedus, Dean & Zoph (2022), Fig. 1.

What is a Mixture of Experts (MoE)?

► Definition

- It combines multiple specialized models, called experts.
- Each expert focuses on a specific subset of data or task.
- This specialization enables efficient and effective learning.

► How does it work?

- MoE uses a gating mechanism.
- The gate selects which expert(s) to use for a given input.
- The model adapts its behavior dynamically based on the input.
- It draws on the strengths of different experts.

► Why is it important?

- MoE models handle large-scale problems efficiently.
- They distribute workload across multiple experts.
- Combining expertise of different models improves performance on complex tasks.

► Challenges

- Training MoE models is computationally expensive.
- Multiple experts need to be trained.
- Designing effective gating mechanisms is difficult.
- Ensuring balanced expert utilization is challenging.

Examples of Mixture of Experts Models

► Switch Transformer

- Uses a gating mechanism to select one or a few experts per input.
- Activates only a small subset of experts at a time.
- Enables large-scale models without excessive computational costs.
- *Example:*
 - Handles language modeling and translation.
 - Uses specialized experts for different aspects of the task.

What is a Mixture of Experts (MoE)? (cont.)

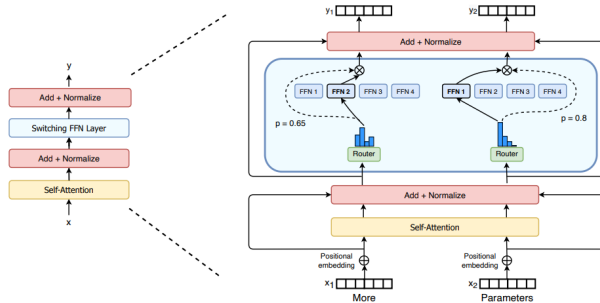


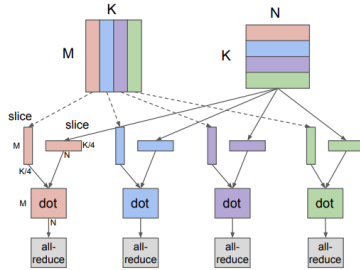
Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens ($x_1 = \text{"More"}$ and $x_2 = \text{"Parameters"}$ below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Source: Fedus et al. (2021)

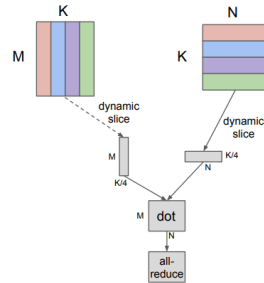
► GShard

- Scales up the number of experts for very large datasets and models.
- Distributes computation across multiple devices.
- Enables efficient training of large models.
- *Example:*
 - Trained a 600B-parameter MoE Transformer for machine translation from 100 languages into English.
 - Utilizes many experts to process different data parts.

What is a Mixture of Experts (MoE)? (cont.)



(a) MPMD Partition



(b) SPMD Partition

Figure 2: Comparison between MPMD and our proposed SPMD partitioning of a Dot operator ($[M, K] \times [K, N] = [M, N]$) across 4 devices. In this example, both operands are partitioned along the contracting dimension K , where each device computes the local result and globally combines with an AllReduce. MPMD partitioning generates separate operators for each device, limiting its scalability, whereas SPMD partitioning generates one program to run on all devices. Note that the compilation time with our SPMD partitioning is not-dependent of the number of devices being used.

GShard's SPMD (single program, multiple data) partitioner splits a single operator across devices, so compilation cost stays independent of how many devices are used. Source: [Lepikhin et al. \(2020\)](#), Fig. 2.

► MoE in BERT

- BERT can be extended with MoE layers.
- MoE layers enhance performance on specific tasks.
- Allows BERT to dynamically select experts based on input.
- Improves ability to handle diverse tasks.
- *Example:*
 - MoE-enhanced BERT performs better on sentiment analysis and question answering.
 - Uses specialized experts for different input types.

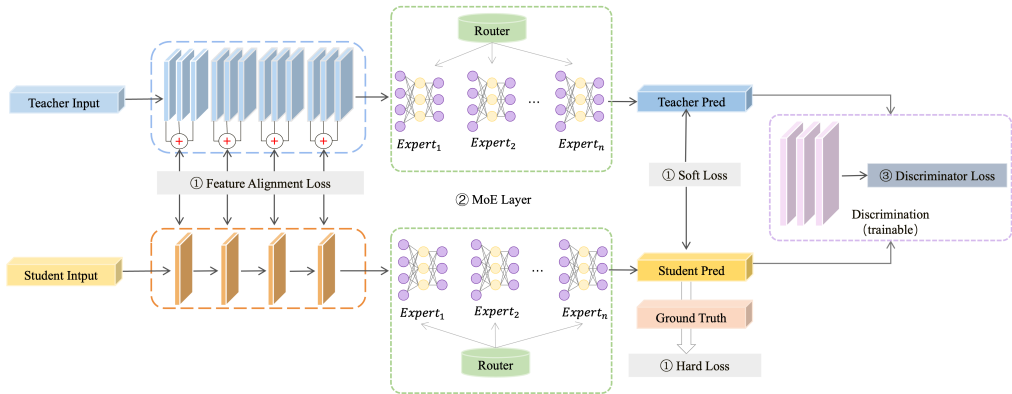
► MicroBERT: Distilling MoE-Based Knowledge

- Distills a full-size BERT teacher into a much smaller student.
- The distillation signal is a sentence-level feature alignment loss guided by a Mixture-of-Experts layer.
- A discriminator trained adversarially pushes the student's predictions towards the teacher's.

What is a Mixture of Experts (MoE)? (cont.)

- Targets resource-constrained environments.
- *Key Points:*
 - ▶ Teacher: BERT; Student: MicroBERT. The MoE sits in the distillation path, not in the teacher backbone.
 - ▶ Reported to perform about 2.24% better than TinyBERT on GLUE at a reduced model size.
 - ▶ Reduces inference time and memory usage.
 - ▶ Suitable for deployment on edge devices.

What is a Mixture of Experts (MoE)? (cont.)

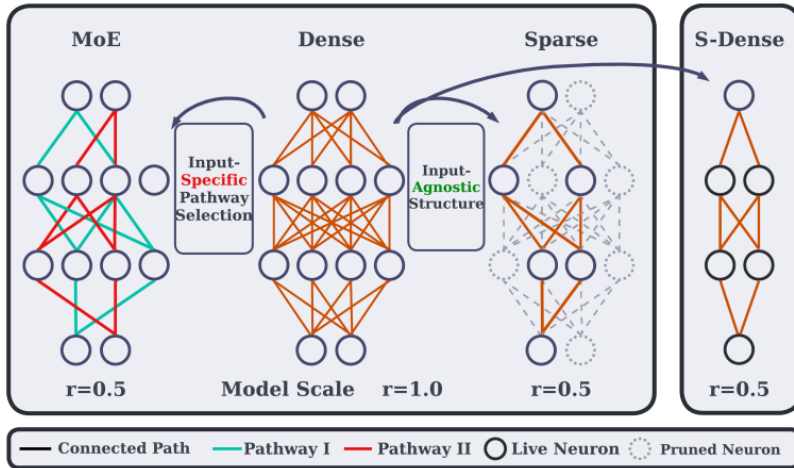


Source: Zheng et al. (2024)

► Other MoE Models

- MoE variants of these families exist: the Switch Transformer is built on T5, and GLaM matches GPT-3 quality using an MoE architecture.
- MoE enables dynamic expert selection based on input.
- Improves learning efficiency and effectiveness.

Sparse vs. Dense Models: **Trade-offs & Motivations**



Source: Zhang et al. (2023)

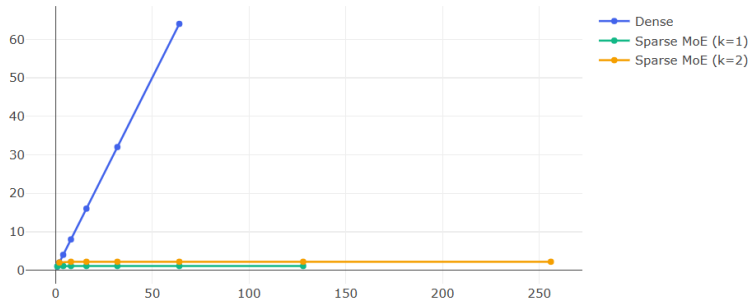
► Dense Models:

- Use a large number of parameters.
- All parameters participate for every input.
- Compute cost is predictable and uniform.
- Training is straightforward: no routing or selection needed.
- Require more computational resources.
- **Examples:**
 - Standard Transformer models (e.g., BERT, GPT).
 - Fully-connected neural networks.

► Sparse Mixture-of-Experts (MoE) Models:

- Only a subset (top- k) of experts are activated per input.
- Enables scaling to billions of parameters without increasing per-token compute.
- Requires a routing mechanism to select experts.
- Focus on specific features or patterns in the data.
- Efficient for large datasets with sparse representations.
- **Examples:**
 - Switch Transformer (Google).
 - GLaM (Google).
 - M6-T (Alibaba).

Sparse vs. Dense Models: Trade-offs & Motivations (cont.)



Comparison showing how FLOPs per token scale with total model parameters. Dense models exhibit coupled growth, while sparse MoE models allow parameter scaling with nearly constant FLOPs per token (assuming fixed k). Note the slight increase for MoE accounts for gating overhead and assumes experts scale parameters. Axes represent relative scale.

Source: **Contrasting Dense vs. Sparse Activation**

► Trade-offs:

- **Dense:** Simpler, stable, but limited by compute and memory.
- **Sparse:** Higher capacity, efficient compute, but adds routing overhead.
- Sparse models may face instability and load balancing issues.
- Choice depends on scale, task complexity, and infrastructure.

Gating Mechanisms – **How Experts Are Selected**

► Gating Mechanism:

- A small neural network that decides which expert(s) to activate for a given input.
- Uses a learned function to compute a score for each expert.
- The scores are used to select the top- k experts.

► How it works:

- Input is passed through a gating network.
- The gating network outputs a probability distribution over experts.
- Only the experts with the highest probabilities are activated.
- This allows the model to focus on relevant experts for each input.

- **MoE Formulation**

- ▶ Input token: x
- ▶ Gating network G calculates expert distribution.

- **Gating Network Calculation**

- ▶ Linear transformation: $logits = W_g x$
- ▶ Softmax activation: $P = \text{softmax}(logits)$
- ▶ W_g : trainable weight matrix
- ▶ P : probability vector for each expert

- **Dense MoE Output**

- ▶ Output $y = \sum_{i=1}^N P_i E_i(x)$
- ▶ Combines all experts with weights P_i .

- **Sparse MoE Goal**

- ▶ Focus on computational efficiency.
- ▶ Only activate a small subset of experts per token.
- ▶ Requires a mechanism to select experts based on P .

- **Prevalent Strategy**

- ▶ For sparse MoEs, selects a subset of experts.
- ▶ Gating network chooses k experts with highest probabilities.
- ▶ Typically, k is small (e.g., 1 or 2).

- **Process Overview**

- ▶ **Compute Scores:** Calculate gating logits or probabilities (P).
- ▶ **Select Top-k:** Identify indices $I = \{i_1, i_2, \dots, i_k\}$ for top k values in P .
- ▶ **Re-normalize Scores (Optional):** take P_{sel} , the sub-vector of P for selected indices I , and re-normalize:

$$P'_j = \frac{\exp(\text{logits}_{i_j})}{\sum_{l=1}^k \exp(\text{logits}_{i_l})} \quad \text{for } j = 1 \dots k$$

- ▶ **Compute Weighted Output:** $y = \sum_{j=1}^k P'_j E_{i_j}(x)$, using the (re-normalized) probabilities as weights.

- **Choice of k**

- ▶ $k = 1$: Token routed to a single expert; maximizes sparsity but limits knowledge combination.
- ▶ $k = 2$: Token uses two experts; balances sparsity and representational capacity at double the computational cost of $k = 1$.
- ▶ k is a key hyperparameter affecting performance and cost.

Gating Mechanisms – How Experts Are Selected (cont.)



Source: Designing Effective Gating Networks

► Noisy Top- k Gating (Shazeer et al., 2017)

- Adds learned noise to the gating scores before the top- k cut is taken:

$$H(x)_i = (xW_g)_i + \varepsilon_i \cdot \text{Softplus}((xW_{\text{noise}})_i), \quad \varepsilon_i \sim \mathcal{N}(0, 1)$$

$$G(x) = \text{softmax}(\text{KeepTopK}(H(x), k)),$$

$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i, & v_i \text{ among the } k \text{ largest} \\ -\infty, & \text{otherwise} \end{cases}$$

- Both W_g and W_{noise} are trained, so the model learns how much noise each expert's score should carry.
- Sending the discarded scores to $-\infty$ makes their softmax weights exactly zero, which is what turns a dense gate into a sparse one.
- The noise exists for load balancing: an expert that starts slightly behind would otherwise never be selected, never train, and stay behind. Noise lets tokens that narrowly missed the cut reach it often enough to break that loop.

► Load-Balanced + Entropy-Regularized Gating

- Ensures efficient and equal expert utilization.
- Prevents overloaded or underutilized experts.
- **Load-Balanced:**
 - Penalty in loss encourages even token distribution.
 - Avoids “expert collapse.”
- **Entropy-Regularized:**
 - Promotes diverse expert selection.
 - Nudges model to use full range of experts.

► Examples in Practice

- **Example 1: Switch Transformer Gating**

- Primarily uses **Top-1 routing**.
- Incorporates **load balancing penalty**.
- Ensures experts are actively learning and distributed.

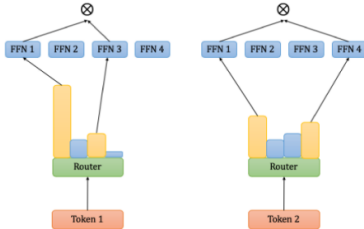
- **Example 2: LLaMA-MoE**

- Employs **Top-2 gating**.
- **Avoids token dropout**.
- Aims for smoother and consistent performance.

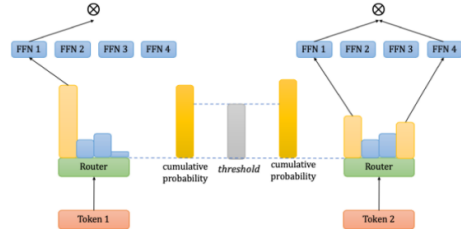
Routing in MoE

► Key Concepts:

- Assign tokens to experts per layer.
- Routing can be token-level or batch-level.
- Penalize overused experts (using load balancing loss).
- Efficient hardware utilization is key.



(a) Top-K routing



(b) Top-P routing

Figure 1: Comparison between Top-K routing mechanism and Top-P routing mechanism. (a) Each token selects fixed $K=2$ experts with Top-K routing probabilities. (b) In Top-P routing mechanism, each token selects experts with higher routing probabilities until the cumulative probability exceeds threshold.

Source: [Huang et al. \(2024\)](#)

► Example 1: DeepSpeed-MoE

- Optimizes distributed routing and communication.
- Achieves linear scaling to hundreds of experts.
- Focuses on efficient data movement across devices.

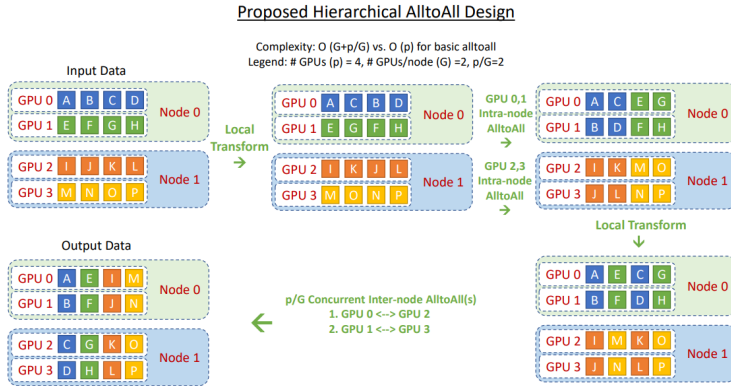


Figure 8: Illustration of the proposed hierarchical all-to-all design.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

Parallelism-coordinated AlltoAll optimization

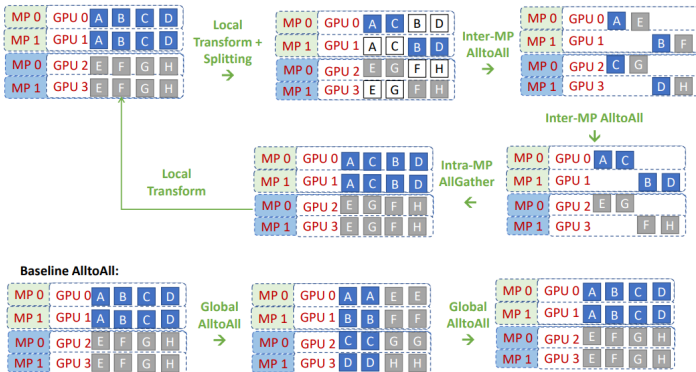


Figure 9: Illustration of the parallelism coordinated communication.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

► Example 2: Tutel by Microsoft

- Uses custom **CUDA** kernels.
- For highly efficient token routing in MoE.
- Reduces overhead and speeds up computation on GPUs.

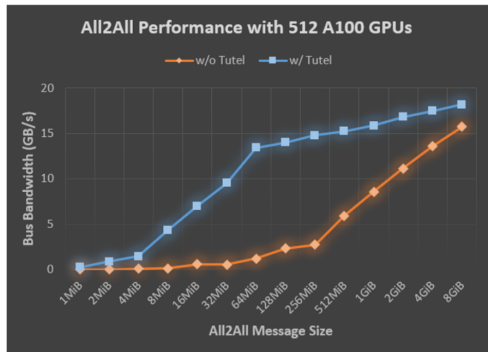
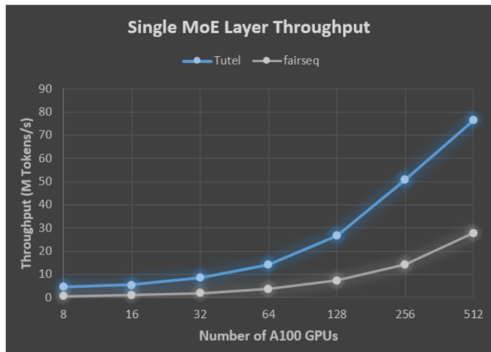


Figure 1 (left): Compared to fairseq, for a single MoE layer, Tutel achieves an 8.49x speedup on an NDm A100 v4 node with 8 GPUs and a 2.75x speedup on 64 NDm A100 v4 nodes with 512 A100 GPUs. The detailed setting is as follows: batch_size = 32, sequence_length = 1,024, Top_K = 2, model_dim = 2,048, and hidden_size = 2,048. Figure 2 (right): The all-to-all bandwidth for different message sizes with 64 NDm A100 v4 nodes (512 A100 GPUs) before and after applying Tutel. Tutel achieves a 2.56x to 5.93x all-to-all speedup with 512 A100 GPUs for message sizes hundreds of MiB large.

Source: Tutel (2021)

MoE at Scale: Load Balancing, Switch Transformers, and Parallelism

- ▶ Switch Transformer (Fedus et al., 2021): Efficient routing and scaling.
- ▶ GShard (Lepikhin et al., 2020): Large-scale MoE for multilingual translation.
- ▶ M6, GLaM, and recent GPT-MoE variants: Further advances in scaling and performance.

Problem: Without intervention, the gating network tends to route most tokens to a few experts, leading to inefficient training and poor expert utilization.

- ▶ **Expert Overload:** Popular experts receive more tokens, train faster, and become even more favored.
- ▶ **Underutilization:** Other experts receive few or no tokens, limiting their contribution.

Solution: Introduce an auxiliary loss to encourage balanced token distribution across experts.

- ▶ **Auxiliary Loss:** Penalizes uneven assignment of tokens, pushing the gating network to distribute tokens more uniformly.
- ▶ **Expert Capacity:** Each expert is assigned a maximum number of tokens it can process (capacity). Excess tokens are dropped or rerouted.
- ▶ **Implementation:** In transformer-based MoEs, this is often controlled via an `aux_loss` parameter.

Auxiliary Loss (Switch Transformer, Eq. 4): for a batch $\mathcal{B}$ of T tokens routed over N experts,

$$L_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^N f_i P_i, \quad f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbf{1}\{\arg \max p(x) = i\}, \quad P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x)$$

- ▶ f_i is the fraction of tokens *dispatched* to expert i ; P_i is the fraction of router *probability* given to expert i ; $p_i(x)$ is the router's softmax score.
- ▶ f_i comes from an arg max, so it carries no gradient. The router learns through P_i , which is differentiable: the loss pushes probability mass away from experts that are already receiving many tokens.
- ▶ The dot product $\sum_i f_i P_i$ is minimised when both vectors are uniform, which is exactly an even split of tokens across experts.
- ▶ The factor N keeps the loss on the same scale as the number of experts changes; Fedus et al. use $\alpha = 10^{-2}$.

Benefits:

- ▶ Ensures all experts are trained and utilized.
- ▶ Improves model efficiency and generalization.
- ▶ Prevents bottlenecks and overload in popular experts.

Transformers and Scaling: Transformers demonstrate that increasing the number of parameters improves performance. Google's GShard project explored scaling transformers beyond 600 billion parameters using MoE layers.

- ▶ **GShard Approach:** Replaces every other feed-forward (FFN) layer with an MoE layer using top-2 gating in both encoder and decoder.
- ▶ **Distributed Training:** MoE layers are shared across devices, while other layers are replicated, enabling efficient large-scale computation.

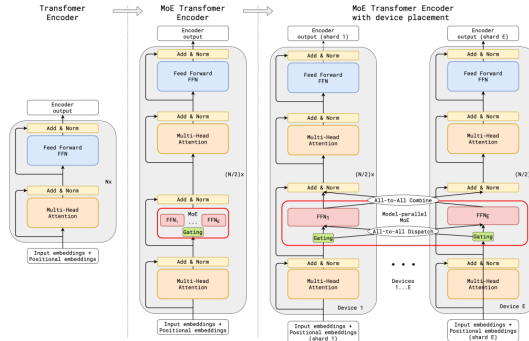


Figure 3: Illustration of scaling of Transformer Encoder with MoE Layers. The MoE layer replaces the every other Transformer feed-forward layer. Decoder modification is similar. (a) The encoder of a standard Transformer model is a stack of self-attention and feed forward layers interleaved with residual connections and layer normalization. (b) By replacing every other feed forward layer with a MoE layer, we get the model structure of the MoE Transformer Encoder. (c) When scaling to multiple devices, the MoE layer is sharded across devices, while all other layers are replicated.

Source: [Lepikhin et al. \(2020\)](#), Fig. 3.

Load Balancing and Efficiency:

- ▶ **Random Routing:** In top-2 gating, the top expert is always selected, while the second expert is chosen probabilistically based on its gating weight.
- ▶ **Expert Capacity:** Each expert has a maximum token capacity. Overflow tokens are either sent via residual connections or dropped, ensuring static tensor shapes and balanced computation.

Why Expert Capacity?

- ▶ Tensor shapes must be fixed at compile time, but token distribution to experts is dynamic. Capacity factors ensure compatibility and efficiency.

Inference Efficiency:

- ▶ During inference, only a subset of experts is activated per input.
- ▶ Shared computations (e.g., self-attention) are applied to all tokens, so the effective compute is much lower than the total parameter count.
- ▶ Example: Mixtral 8x7B has about 47B total parameters but activates only about 13B per token with top-2 gating, so its inference compute is close to that of a 13B dense model.

Switch Transformers: Address training and fine-tuning instabilities in MoEs by simplifying routing and improving efficiency. The authors released a 1.6T parameter MoE model with 2048 experts on Hugging Face, achieving a 4x pre-train speed-up over T5-XXL.

- ▶ **Switch Transformer Layer:** Replaces FFN layers with MoE layers, but routes each token to a single expert (single-expert strategy), reducing router computation and communication costs while preserving quality.
- ▶ **Expert Capacity:** Defined as

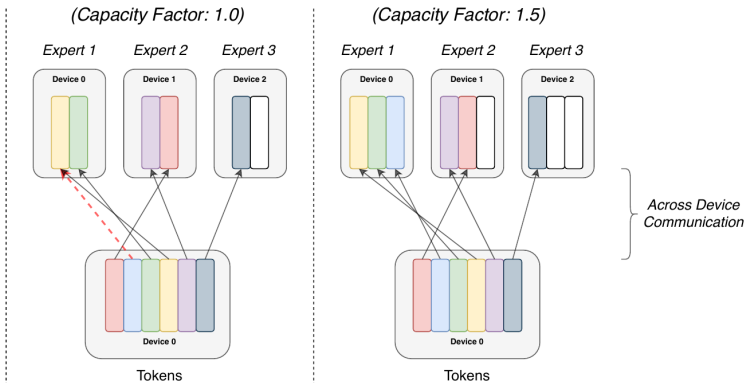
$$\text{Expert Capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}$$

- ▶ Low capacity factors (1–1.25) are effective, balancing efficiency and communication overhead.
- ▶ **Load Balancing Loss:** Simplified auxiliary loss encourages uniform routing and is weighted by a hyperparameter.

- **Selective Precision:** Experts are trained in bfloat16, while routing uses full precision to avoid instability. This reduces communication and computation costs without degrading quality.

Terminology

- **Experts:** Split across devices, each having their own unique parameters. Perform standard feed-forward computation.
- **Expert Capacity:** Batch size of each expert. Calculated as $(\text{tokens_per_batch} / \text{num_experts}) * \text{capacity_factor}$
- **Capacity Factor:** Used when calculating expert capacity. Expert capacity allows more buffer to help mitigate token overflow during routing.



Each expert processes a fixed batch of $(\text{tokens per batch} / \text{experts}) \times \text{capacity factor}$. At a capacity factor of 1.0 an overloaded expert overflows and the extra token is dropped (dashed red); at 1.5 nothing is dropped, at the cost of padded empty slots (white). Source: Fedus et al. (2021), Fig. 3.

Architecture:

- ▶ Encoder-decoder setup, similar to T5 but with MoE layers.
- ▶ Fine-tuning and summarization tasks are supported.

GLaM and Scaling:

- ▶ GLaM explores scaling MoEs further, matching GPT-3 quality with 1/3 the energy.
- ▶ Uses decoder-only models, top-2 routing, and larger capacity factors.
- ▶ Capacity factor can be adjusted during training and inference to control compute and efficiency.

Design: DeepSeek-V3 has 671B total parameters and activates 37B of them per token; DeepSeek-R1 is trained from it and inherits the same architecture. Every feed-forward layer except the first three is an MoE layer holding 1 shared expert and 256 routed experts, of which 8 are activated per token.

- ▶ **Fine-grained segmentation:** each expert is made smaller and more of them are activated. Splitting N experts into mN smaller ones and activating mk instead of k leaves the active parameter count unchanged, but gives far more distinct combinations of experts, so each one can specialise more narrowly.
- ▶ **Shared experts:** a small number of experts are always active for every token. They absorb the knowledge every token needs, which keeps that knowledge from being duplicated inside every routed expert.

Auxiliary-loss-free load balancing:

- ▶ An auxiliary loss balances experts, but it competes with the language-modelling objective: too large a coefficient costs quality.
- ▶ DeepSeek-V3 instead keeps a bias b_i per expert and adds it to the affinity score when choosing the top- k . Overloaded experts have their bias decreased and underloaded experts increased, by a fixed step at each training step.
- ▶ The bias only decides *which* experts are selected. The gating value that multiplies an expert's output is still computed from the original affinity score, so the forward pass is left undistorted.
- ▶ A sequence-wise balance loss is retained with a very small weight ($\alpha = 10^{-4}$), only to rule out extreme imbalance inside a single sequence.

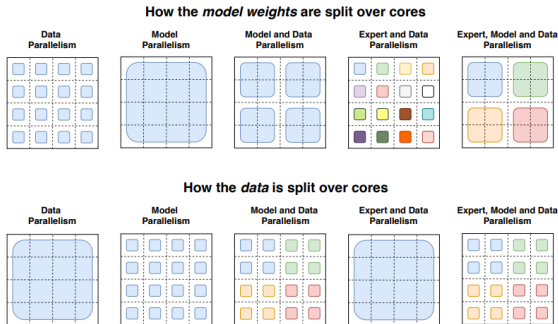
Parallelism Overview:

- ▶ **Data Parallelism:** Replicates model weights across all cores; data is partitioned so each core processes a different batch.
- ▶ **Model Parallelism:** Splits the model across cores; each core holds a part of the model, and data is replicated.
- ▶ **Model and Data Parallelism:** Both model and data are partitioned; different cores process different batches and hold different model segments.
- ▶ **Expert Parallelism:** Experts are distributed across workers. Each worker processes a different batch, and MoE layers route tokens to the appropriate expert on each worker.

Expert Parallelism Details:

- ▶ For non-MoE layers, expert parallelism acts like data parallelism.
- ▶ For MoE layers, tokens are dynamically routed to the worker hosting the selected expert.

- Enables scaling to large numbers of experts and efficient distributed training.



Each 4×4 grid is 16 cores; the top row shows how model weights are split across them and the bottom row how the batch of tokens is split. Source: Fedus et al. (2021), Fig. 9.

Benefits of MoE

► Scalability:

- Can scale to billions of parameters.
- Only a small subset of experts is activated for each input.
- Allows for larger models without proportional increases in compute.

► Efficient Computation:

- Activates only a small fraction of the total parameters per token: roughly k out of N experts, so the fraction shrinks as N grows at fixed k .
- Mixtral 8x7B activates about 13B of 47B parameters ($\sim 28\%$); DeepSeek-V3 activates 37B of 671B ($\sim 5.5\%$).
- Reduces computational cost per token.
- Sparse activation means fewer parameters are active at any time.
- Enables training on larger datasets with limited resources.
- Leads to lower latency and inference cost.
- Allows for models with vastly more parameters.

► Flexibility:

- Different experts can specialize in different tasks or data distributions.
- Allows for more nuanced model behavior and adaptability.
- Can be tailored to specific applications or domains.

► Specialization:

- Experts can focus on specific domains (e.g., math, code, biology, logic).
- Improves accuracy and performance in specialized tasks.

► Example 1: MoE with Domain Experts

- A math expert becomes active only on arithmetic inputs.
- Improves accuracy specifically for math problems.
- Outperforms generalist models on these tasks.

► Example 2: MoRE (Mixture-of-Reasoning-Experts)

- Specialises one backbone model into four reasoning experts by prompting: factual, multihop, mathematical, and commonsense.
- Every expert answers every question and a trained selector picks the best answer, so this buys generality and interpretability rather than the compute saving a routed MoE gives.
- Agreement between experts also tells the system when to abstain.

► Performance:

- Often achieves state-of-the-art results on various benchmarks.
- Combines the strengths of multiple models without the overhead of training them separately.
- Can outperform dense models with fewer active parameters.

Limitations of MoE Models

- ▶ **Training instability:** Some experts may be underused, leading to “expert collapse.”
- ▶ This is often addressed with load balancing.
- ▶ **Hard to debug:** Expert specialization may not always align with human-interpretable domains.
- ▶ Makes understanding model behavior challenging.
- ▶ **Distributed training cost:** Load balancing across GPUs adds significant engineering overhead.
- ▶ Requires complex distributed systems.

► Example 1: GShard training instability

- Some experts never activate after several epochs of training.
- Requires manual intervention or advanced balancing techniques.

► Example 2: Switch Transformer

- Needed custom gradient balancing.
- To prevent expert overload or dropout during training.

References & Further Reading

- ▶ DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. [arXiv:2501.12948](#).
- ▶ Financial Times. (2025). *Transcript: The story behind DeepSeek's breakthrough*. [ft.com](#).
- ▶ Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. [arXiv:1701.06538](#).
- ▶ Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Chen, Z. (2020). *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*. [arXiv:2006.16668](#).
- ▶ Fedus, W., Zoph, B., & Shazeer, N. (2021). *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. [arXiv:2101.03961](#). JMLR 23(120), 2022.
- ▶ Du, N., Huang, Y., Dai, A. M., et al. (2021). *GLaM: Efficient Scaling of Language Models with Mixture-of-Experts*. [arXiv:2112.06905](#). ICML 2022.

- ▶ Fedus, W., Dean, J., & Zoph, B. (2022). *A Review of Sparse Expert Models in Deep Learning*. [arXiv:2209.01667](#).
- ▶ Rajbhandari, S., Li, C., Yao, Z., et al. (2022). *DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale*. [arXiv:2201.05596](#). ICML 2022.
- ▶ Cui, W., Xiong, Y., Cheng, P., & Salas, R. (2021). *Tutel: An Efficient Mixture-of-Experts Implementation for Large DNN Model Training*. Microsoft Research Blog. [microsoft.com](#).
- ▶ Jiang, A. Q., Sablayrolles, A., Roux, A., et al. (2024). *Mixtral of Experts*. [arXiv:2401.04088](#).
- ▶ Dai, D., Deng, C., Zhao, C., et al. (2024). *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*. [arXiv:2401.06066](#).
- ▶ DeepSeek-AI et al. (2024). *DeepSeek-V3 Technical Report*. [arXiv:2412.19437](#).

- ▶ Shen, S., Hou, L., Zhou, Y., Du, N., Longpre, S., Wei, J., Chung, H. W., Zoph, B., Fedus, W., Chen, X., et al. (2023). *Mixture-of-Experts Meets Instruction Tuning: A Winning Combination for Large Language Models*. [arXiv:2305.14705](#).
- ▶ Zhang, Y., Cai, R., Chen, T., et al. (2023). *Robust Mixture-of-Expert Training for Convolutional Neural Networks*. [arXiv:2308.10110](#). ICCV 2023.
- ▶ Huang, Q., An, Z., Zhuang, N., et al. (2024). *Harder Tasks Need More Experts: Dynamic Routing in MoE Models*. [arXiv:2403.07652](#).
- ▶ Zheng, D., Li, J., Yang, Y., Wang, Y., & Pang, P. C.-I. (2024). *MicroBERT: Distilling MoE-Based Knowledge from BERT into a Lighter Model*. *Applied Sciences*, 14(14), 6171. [mdpi.com](#).
- ▶ Kranen, K., & Nguyen, V. (2024). *Applying Mixture of Experts in LLM Architectures*. NVIDIA Technical Blog. [developer.nvidia.com](#).
- ▶ Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., & Huang, J. (2024). *A Survey on Mixture of Experts in Large Language Models*. [arXiv:2407.06204](#).

- ▶ DataCamp. (2024). *What Is Mixture of Experts (MoE)? How It Works, Use Cases & More*. datacamp.com.
- ▶ Si, C., et al. (2023). *Getting MoRE out of Mixture of Language Model Reasoning Experts*. In *Findings of EMNLP 2023*. [ACL Anthology](#).